Resource Model v0.1 Amendment

1. Device Library and Resource Library

Device Library describes what a Device is:

```
use standard.lib;
```

Resource Library describes how a Device may be presented:

```
use standard.resource;
```

The two libraries have separate responsibilities.

```
standard.lib
    Device identity
    ports
    pins
    package
    behavior
    timing
    simulation

standard.resource
    symbols
    DIP visuals
    breadboard visuals
    icons
    labels
    colors
    animations
    terminal widgets
    display widgets
```

---

2. Component Example

```
component:component CounterDemo {
    use standard.lib;
```

```
    device Clock is OSCILLATOR;
    device Counter is 74HC161;
    device R[4] is RESISTOR;
    device LED[4] is LED;

    Counter[Q0, Q1, Q2, Q3]
        as Counter.Q[0..3];

    connect Clock.OUT -> Counter.CLK;
    connect Counter.Q[0..3] -> R[0..3].1;
    connect R[0..3].2 -> LED[0..3].A;
    connect LED[0..3].K -> GND;
}
```

`component:component` depends on `standard.lib` because it needs Device meaning and simulation behavior.

---

3. Board Example

```
component:board MainBoard {
    use CounterDemo;

    use standard.resource;

    place Clock at 80, 180;
    place Counter at 320, 180;
    place R[0..3] at 560, 100;
    place LED[0..3] at 760, 100;

    show port_name;
    show pin_number;
    show signal_value;
}
```

`component:board` depends on `standard.resource` because it needs visual resources.

---

4. Multiple Resource Libraries

A Board may use more than one Resource Library:

```
component:board MainBoard {
    use CounterDemo;
```

```
    use standard.resource;
    use breadboard.resource;
    use classroom.resource;
}
```

Possible roles:

```
standard.resource
    default visual resources

breadboard.resource
    breadboard-oriented Devices and wiring

schematic.resource
    schematic symbols

classroom.resource
    larger labels and teaching annotations

retro.resource
    visual style for historical computers

rv8.resource
    custom visual resources for RV8
```

---

5. Device-to-Resource Mapping

A Resource Library maps a Device type from a Device Library to one or more visual representations.

Conceptual mapping:

```
standard.lib::74HC161
    ↓
standard.resource::74HC161.default

standard.lib::74HC161
    ↓
breadboard.resource::74HC161.dip16

standard.lib::74HC161
```

```
    ↓
schematic.resource::74HC161.logic_symbol
```

A Resource Library does not redefine the Device.

It only describes how the Device is presented.

---

6. Resource Selection

A Board may select a default presentation mode:

```
component:board MainBoard {
    use CounterDemo;
    use standard.resource;
    use breadboard.resource;

    view breadboard;
}
```

Or select a resource for a specific Device:

```
show Counter as breadboard.resource.74HC161.dip16;
show Clock as standard.resource.OSCILLATOR.compact;
```

The exact syntax for explicit resource selection remains provisional, but the model is frozen:

> Resource Libraries provide presentation alternatives for Devices.

---

7. Resource Resolution Order

When several Resource Libraries provide a visual for the same Device, Board resolves them in declaration order.

Example:

```
use standard.resource;
use classroom.resource;
```

Possible rule:

```
Later Resource Libraries override earlier ones.
```

Therefore:

```
standard.resource
    provides default 74HC161 visual

classroom.resource
    replaces it with a teaching-oriented visual
```

However, explicit selection must override automatic resolution.

Recommended precedence:

```
1. Explicit Device resource selection
2. Last matching Resource Library
3. Earlier matching Resource Library
4. Device Library fallback metadata
5. Generic unknown-Device visual
```

---

8. Mapping Compatibility

A Resource mapping should identify the Device Library and Device type it supports.

Conceptual metadata:

```
{
  "resource": {
    "name": "74HC161.dip16",
    "maps": "standard.lib::74HC161",
    "view": "breadboard",
    "package": "DIP16"
  }
}
```

The Resource Resolver must verify:

- Device type identity
- package compatibility
- expected port names

- physical pin mapping
- optional resource version compatibility

A Resource Library must not silently change port or pin meaning.

---

9. One Device, Many Resources

A single Device may have several visual forms:

```
74HC161
├── compact block
├── logic symbol
├── DIP16 top view
├── breadboard view
├── pinout view
├── teaching view
└── debug view
```

These are projections of the same Device instance.

Switching resources must not change:

- Device identity
- ports
- physical pins
- connectivity
- simulation state

---

10. Virtual Device Resources

Virtual Devices also use Resource Libraries.

Examples:

```
TERMINAL
    terminal widget

PROBE
    probe marker

OSCILLATOR
    clock control widget
```

```
DISPLAY
    display surface

LOGIC_ANALYZER
    waveform panel

VCC
    power symbol

GND
    ground symbol
```

The behavior remains in the Device Library.

The interactive visual representation belongs to the Resource Library.

---

11. Interactive Resources

A Resource may describe interaction, but it must not own Device behavior.

Example:

```
OSCILLATOR Device
    behavior:
        standard.lib

OSCILLATOR clock knob and step button
    interaction and visual:
        standard.resource
```

Board interaction produces Operations:

```
User presses clock button
    ↓
Resource reports interaction
    ↓
Board sends Operation
    ↓
Components updates OSCILLATOR Device
    ↓
Board receives new state
```

Resource code must not directly mutate simulation state.

---

12. Resource Library Types

Resource Libraries may contain:

```
Device visuals
package drawings
wire styles
bus styles
fonts
icons
animations
sound
terminal renderers
display renderers
timeline renderers
waveform renderers
teaching annotations
Board themes
```

A Resource Library may be narrowly focused or comprehensive.

---

13. Package Example

```
CounterDemo/
├── main.component
├── main.board
├── operations/
│   └── noise_test.operation
└── resources.lock
```

`main.component`:

```
component:component CounterDemo {
    use standard.lib;
}
```

`main.board`:

```
component:board MainBoard {
    use CounterDemo;

    use standard.resource;
    use breadboard.resource;
}
```

`resources.lock` may record resolved versions:

```
{
  "resources": {
    "standard.resource": "0.1.0",
    "breadboard.resource": "0.1.0"
  }
}
```

---

14. Ownership

```
Device Library
    owns Device meaning

Resource Library
    owns reusable presentation resources

Component
    owns machine topology

Board
    owns placement and presentation choices

Components
    owns resolution, validation, and execution
```

---

15. Frozen v0.1 Rule

The following forms are accepted:

```
component:component CounterDemo {
    use standard.lib;
}
```

```
component:board MainBoard {
    use CounterDemo;

    use standard.resource;
}
```

Multiple Resource Libraries are allowed:

```
use standard.resource;
use breadboard.resource;
use classroom.resource;
```

Resource Libraries may map Device types from one or more Device Libraries, but they must not redefine Device identity, ports, pins, behavior, or simulation semantics.